Homework 2 – Artificial Intelligence

Neural Models for Image Classification

Susai Florian – Group 332CA

May 2026

Contents

1 Introduction **3**

2 Data Augmentation Visualization (Requirement 3.1) **3**
2.1 Augmentation Strategy . . . 3
2.2 Visualization . . . 3

3 MLP Architecture (Requirement 3.2) **5**
3.1 Architecture Description . . . 5
3.2 Training Configuration . . . 6
3.3 MLP Results . . . 6
3.3.1 Training Curves . . . 6
3.3.2 Confusion Matrices . . . 7

4 CNN Architecture (Requirement 3.3) **7**
4.1 Architecture Description . . . 7
4.2 Training Configuration . . . 8
4.3 CNN Results . . . 8

5 Fine-tuning with ResNet-18 (Requirement 3.4) **9**
5.1 Fine-tuning Procedure . . . 9
5.2 Training Configuration . . . 9
5.3 Fine-tuning Results . . . 10

6 Ablation Studies (Requirement 3.5) **10**
6.1 MLP: Impact of Dropout (Requirement 3.5.1) . . . 11
6.2 CNN: Impact of BatchNorm (Requirement 3.5.2) . . . 11
6.3 CNN: Impact of Data Augmentation (Requirement 3.5.3) . . . 12
6.4 Class Imbalance Handling Techniques (Requirement 3.5.4) . . . 13
6.5 Fine-tuning: Learning Rate Warmup (Requirement 3.5.5) . . . 14

7 Curriculum Learning (BONUS) **15**
7.1 Methodology . . . 15
7.2 Results . . . 16

8 Comparative Summary of All Experiments **17**

9 Kaggle Submissions **18**

10 Implementation Details **18**
10.1 Computational Complexity 18
10.2 Infrastructure 18
10.3 Reproducibility 19

1 Introduction

This report documents the implementation, training, and evaluation of three types of neural architectures (MLP, CNN, fine-tuned ResNet-18) on two image classification problems: skin lesion recognition (*Vai de pielea mea*, 7 classes, 6009 training images) and facial expression recognition (*You're on candid camera*, 7 classes, 10434 training images). Both datasets are imbalanced, which makes F1-macro a more informative metric than accuracy.

The complete pipeline was implemented in PyTorch and includes: custom `Dataset` classes, dataset-specific augmentations, training with `CosineAnnealingLR`, TensorBoard logging, and systematic ablation studies. All experiments were run on an NVIDIA GTX 1070 Ti GPU (8 GB VRAM).

The source code is in `solution.py`, and all results (figures, confusion matrices, Kaggle submissions) are saved in the `results/` directory.

2 Data Augmentation Visualization (Requirement 3.1)

2.1 Augmentation Strategy

Augmentations were chosen based on the specific characteristics of each dataset:

Skin Lesions (skin):

- **Horizontal + Vertical Flip** – dermoscopic images have no fixed natural orientation, so all flips are valid.
- **Random Rotation (up to 180°)** – moles can appear at any angle on the skin surface.
- **Subtle ColorJitter** (brightness=0.15, contrast=0.15, saturation=0.1, hue=0.05) – simulates slight variations in imaging conditions; values are kept small to preserve diagnostically relevant color information.
- **RandomCrop + Resize** – simulates different zoom levels and positioning.

Facial Expressions (face):

- **Horizontal Flip** – faces are approximately symmetric.
- **Random Rotation (±15°)** – simulates slight head tilts.
- **No Vertical Flip** – faces do not appear upside down in real scenarios.
- **ColorJitter** (brightness=0.2, contrast=0.2, saturation=0.2, hue=0.1) – simulates different lighting conditions; more aggressive than skin since color is not the primary classification feature.
- **GaussianBlur** (kernel 3) and **RandomCrop + Resize**.

2.2 Visualization

Figures 1 and 2 show augmentation examples applied to images from each class.

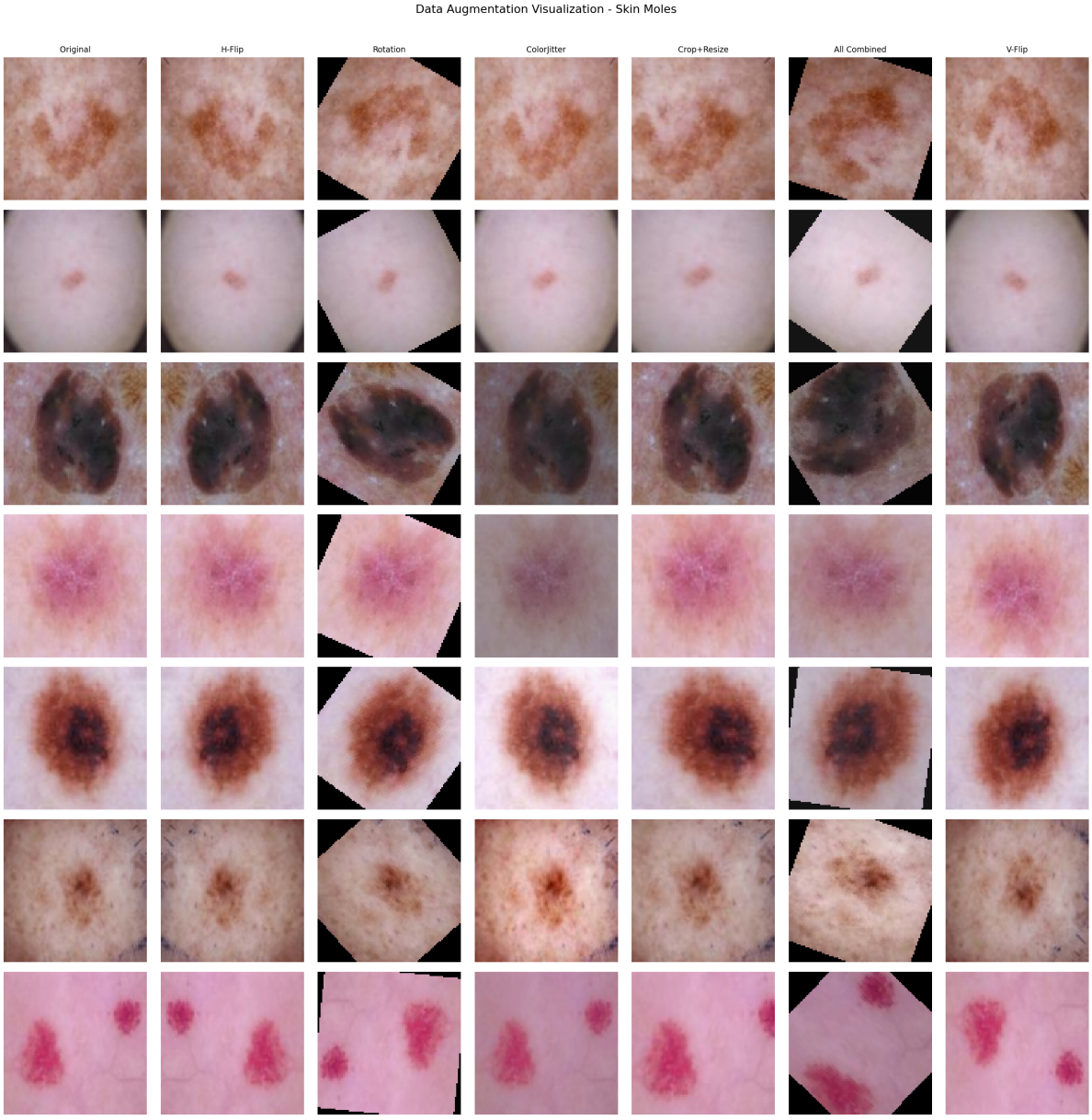


Figure 1: Augmentation visualization on the *skin moles* dataset. Each row contains an original image and its augmented variants (flip, rotation, jitter, crop).

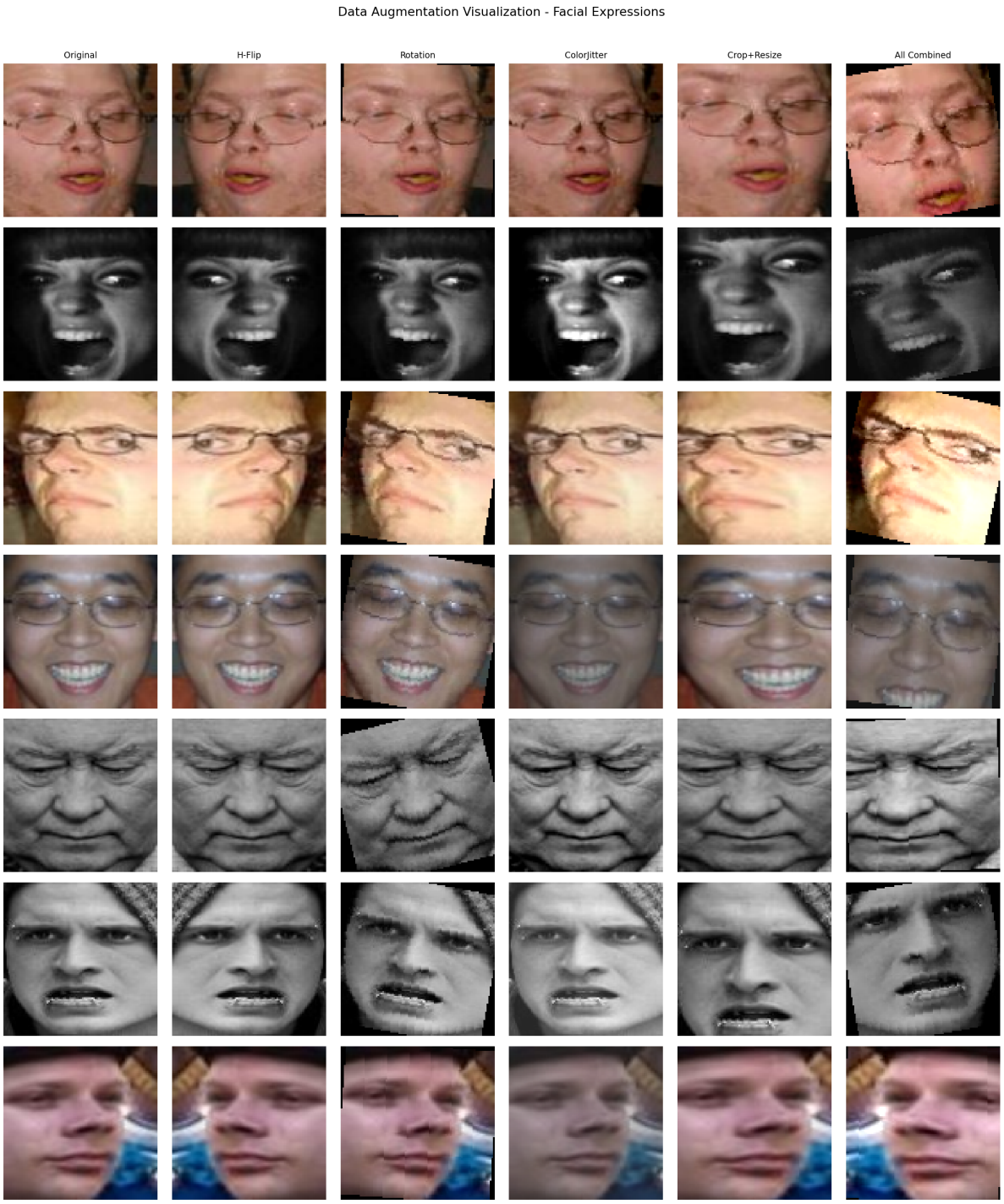


Figure 2: Augmentation visualization on the *facial expressions* dataset. Geometric augmentations are more conservative (no vertical flip, rotation $\pm 15°$).

3 MLP Architecture (Requirement 3.2)

3.1 Architecture Description

The MLP architecture flattens the image into a vector and processes it through three fully connected hidden layers:

$$\texttt{Flatten} \rightarrow [\text{Linear} \rightarrow \text{BN1d} \rightarrow \text{ReLU} \rightarrow (\text{Dropout})] \times 3 \rightarrow \texttt{Linear(128, } C\texttt{)}$$

Table 1: Detailed MLP architecture

Layer	Input Dim	Output Dim	Components
Flatten	$3 \times 64 \times 64$	12 288	–
Hidden 1	12 288	512	Linear + BN1d + ReLU + Dropout(0.3)
Hidden 2	512	256	Linear + BN1d + ReLU + Dropout(0.3)
Hidden 3	256	128	Linear + BN1d + ReLU + Dropout(0.3)
Output	128	C (7)	Linear

Design justifications:

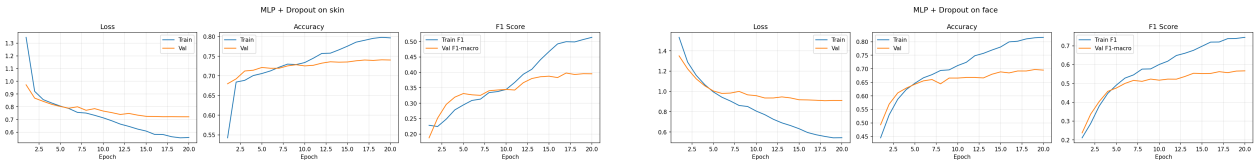
- Decreasing layer dimensions ($512 \to 256 \to 128$) perform a gradual information compression, forcing the network to learn increasingly abstract representations.
- **BatchNorm1d** normalizes activations per mini-batch, stabilizing training and accelerating convergence.
- **Dropout(0.3)** prevents overfitting on the training set; its effect is evaluated in the ablation study in Section 6.1.
- **ReLU** activation functions provide the necessary non-linearity.

3.2 Training Configuration

- **Optimizer:** Adam (lr $= 10^{-3}$, weight decay $= 10^{-4}$)
- **Scheduler:** CosineAnnealingLR ($T_{max} = 20$)
- **Loss function:** CrossEntropyLoss
- **Epochs:** 20
- **Batch size:** 256
- **Image size:** 64×64 (no augmentations)

3.3 MLP Results

3.3.1 Training Curves



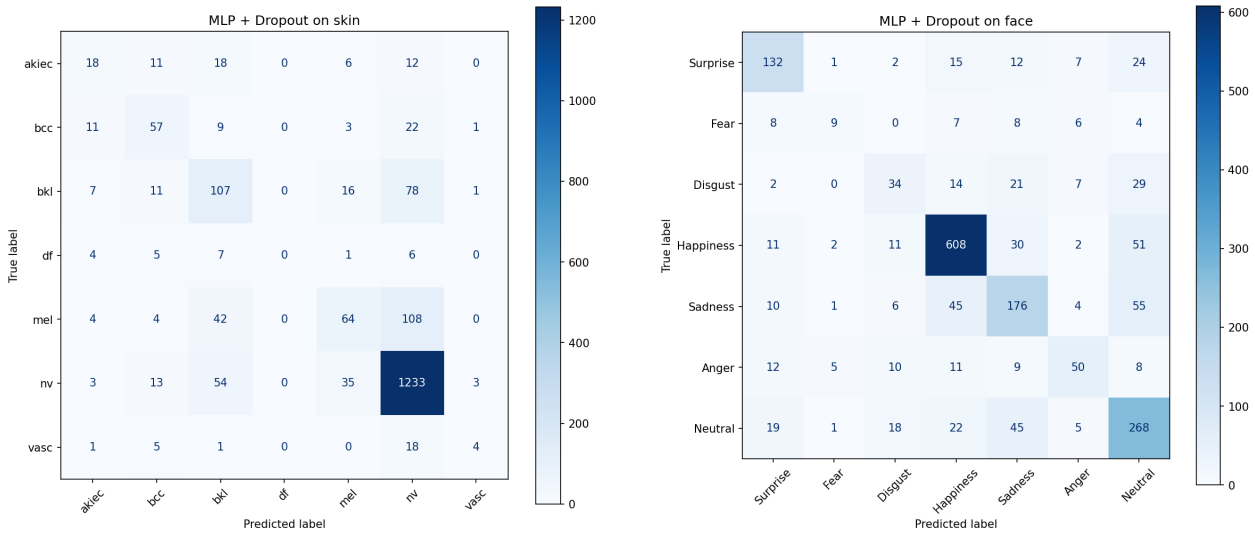
(a) Skin – MLP with Dropout

(b) Face – MLP with Dropout

Figure 3: Training curves (loss and accuracy on train/val) for the MLP with dropout configuration.

On both datasets, the training loss decreases steadily while the validation loss plateaus, indicating mild overfitting. The train/val gap is more pronounced on the *face* dataset due to its larger size and higher variability.

3.3.2 Confusion Matrices



(a) Skin – MLP with Dropout

(b) Face – MLP with Dropout

Figure 4: Confusion matrices for MLP with Dropout on both datasets.

On the *skin* dataset, the confusion matrix reveals that the MLP predominantly predicts the majority class *nv* (Melanocytic Nevi), achieving high accuracy but low F1-macro. Minority classes (*akiec*, *df*, *vasc*) are rarely detected correctly. On the *face* dataset, performance is more balanced, with natural confusions between similar emotions (e.g., *Sadness* vs *Neutral*).

4 CNN Architecture (Requirement 3.3)

4.1 Architecture Description

The CNN adheres to the imposed constraints (at most 4 convolutional layers, at most 2 FC layers):

$$[\text{Conv2d} \to (\text{BN2d}) \to \text{ReLU} \to \text{MaxPool2d}] \times 3 \to [\text{Conv2d} \to (\text{BN2d}) \to \text{ReLU}] \to \texttt{AdaptiveAvgPool2d(1)} \to \texttt{FC} \times 2$$

Table 2: Detailed CNN architecture

Block	Channels	Kernel	Pooling	Components
Conv Block 1	$3 \to 32$	3×3	MaxPool 2×2	Conv + BN2d + ReLU + Pool
Conv Block 2	$32 \to 64$	3×3	MaxPool 2×2	Conv + BN2d + ReLU + Pool
Conv Block 3	$64 \to 128$	3×3	MaxPool 2×2	Conv + BN2d + ReLU + Pool
Conv Block 4	$128 \to 256$	3×3	–	Conv + BN2d + ReLU
GAP	256	–	AdaptiveAvgPool(1)	Global Average Pooling
FC 1	$256 \to 128$	–	–	Linear + BN1d + ReLU + Dropout(0.5)
FC 2	$128 \to C$	–	–	Linear

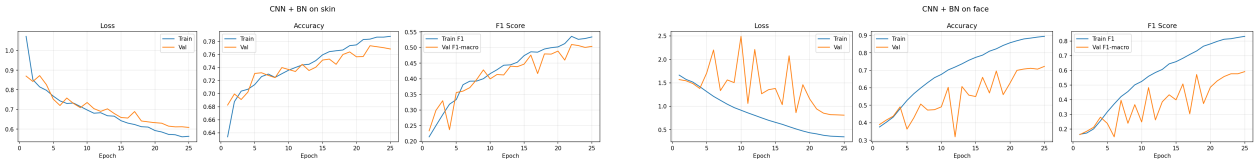
Architectural design justifications:

- 3×3 **kernels throughout:** the receptive field grows sufficiently across the 4 stacked convolutional blocks ($3 \to 5 \to 9 \to 17$ pixels at each level), while smaller kernels have fewer parameters.
- **Increasing channel count** ($32 \to 64 \to 128 \to 256$) allows extraction of increasingly complex features at each depth level.
- **MaxPool** 2×2 on the first 3 blocks gradually reduces spatial dimensions, while the last block omits pooling to preserve feature map resolution before GAP.
- **AdaptiveAvgPool2d(1)** makes the network input-size agnostic, producing a 256-dimensional vector regardless of input resolution.
- **Bias=False** when using BatchNorm2d, since BN includes its own bias parameter (β).
- **Dropout(0.5)** between FC layers regularizes the classifier head.

4.2 Training Configuration

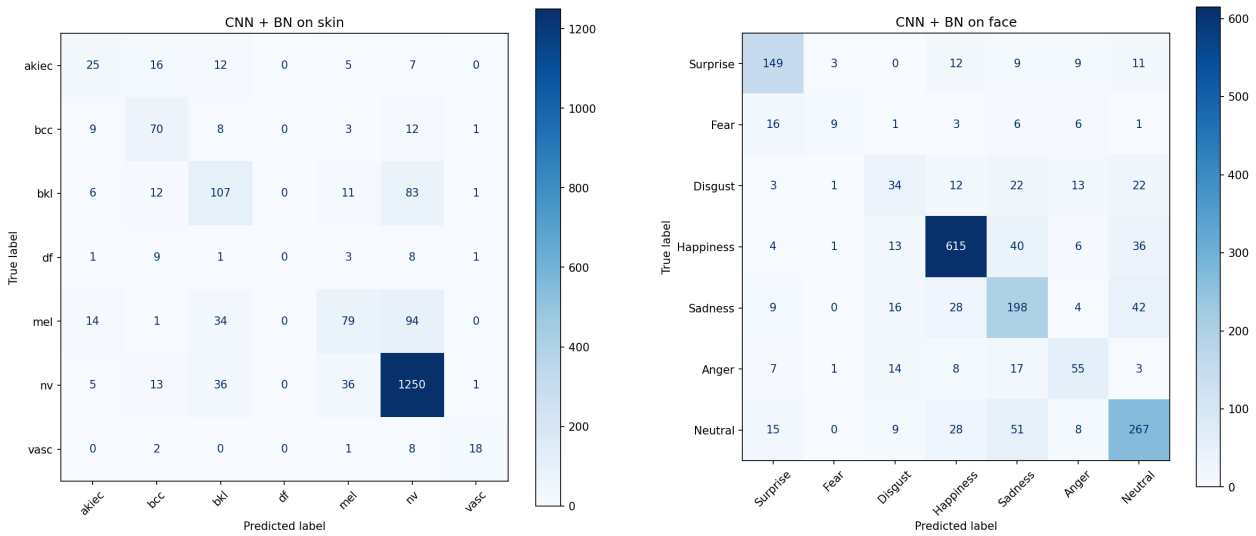
- **Optimizer:** Adam (lr $= 10^{-3}$, weight decay $= 10^{-4}$)
- **Scheduler:** CosineAnnealingLR ($T_{max} = 25$)
- **Loss function:** CrossEntropyLoss
- **Epochs:** 25
- **Batch size:** 128
- **Image size:** 96×96

4.3 CNN Results



(a) Skin – CNN + BatchNorm (b) Face – CNN + BatchNorm

Figure 5: Training curves for CNN with BatchNorm on both datasets.



(a) Skin – CNN + BatchNorm (b) Face – CNN + BatchNorm

Figure 6: Confusion matrices for CNN + BatchNorm.

The CNN with BatchNorm achieves significantly better results than the MLP on both datasets: F1-macro 0.5097 vs 0.3980 on *skin* and 0.5914 vs 0.5668 on *face*. Convolutional layers capture local spatial features that the MLP (which flattens the image) loses entirely. On the *face* dataset, the CNN reaches 72.24% accuracy, demonstrating its ability to distinguish between different facial expressions.

5 Fine-tuning with ResNet-18 (Requirement 3.4)

5.1 Fine-tuning Procedure

We used **ResNet-18** pre-trained on ImageNet (1000 classes, 1.28M images), adapting it for our two 7-class classification problems.

Classification head replacement: The original `fc` layer of ResNet-18 (Linear 512 $\rightarrow$ 1000) was replaced with:

$$\texttt{Linear(512, 256)} \rightarrow \texttt{ReLU} \rightarrow \texttt{Dropout(0.3)} \rightarrow \texttt{Linear(256, 7)}$$

Optimization strategy – Differential learning rates: We used **two parameter groups** with different learning rates:

- **Backbone** (pre-trained convolutional layers): lr $= 10^{-4}$ – a small rate to avoid destroying features learned on ImageNet.
- **Head** (new classifier): lr $= 10^{-3}$ – 10x higher to quickly learn the projection into the 7-class space.

Class imbalance handling: We used **Weighted CrossEntropyLoss**, with weights inversely proportional to each class frequency. *Important:* we did not combine the weighted sampler with weighted loss, as overlapping both mechanisms can push the training imbalance in the opposite direction.

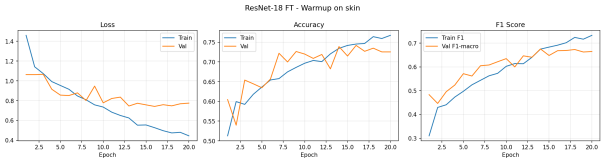
Input adaptation:

- Images are resized to 224×224 (standard ImageNet resolution).
- Normalization uses ImageNet mean and standard deviation ($\mu = [0.485, 0.456, 0.406]$, $\sigma = [0.229, 0.224, 0.225]$).
- Both datasets contain RGB images with 3 channels, so no channel adaptation is needed.

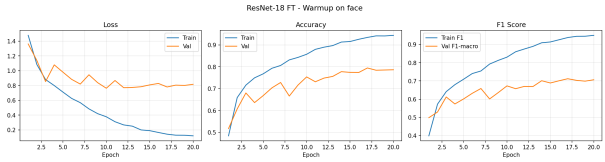
5.2 Training Configuration

- **Optimizer:** Adam (differential LR: backbone 10^{-4}, head 10^{-3}, weight decay 10^{-4})
- **Scheduler:** CosineAnnealingLR ($T_{max} = 20$) or Warmup + Cosine (details in Section 6.5)
- **Loss function:** Weighted CrossEntropyLoss
- **Epochs:** 20
- **Batch size:** 32
- **Augmentation:** full configuration (*all*)

5.3 Fine-tuning Results

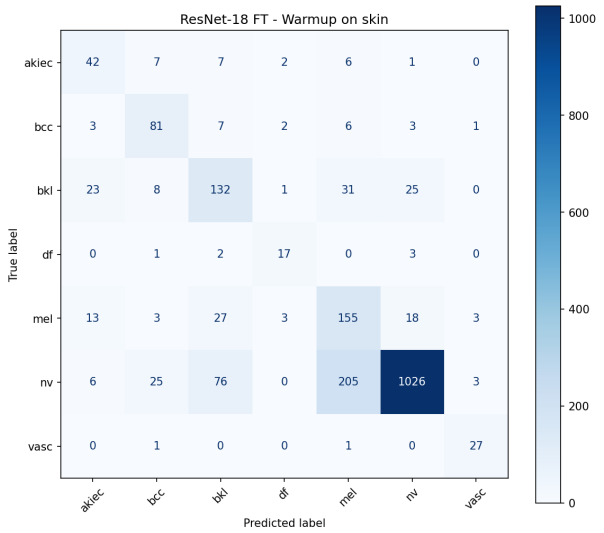


(a) Skin – ResNet-18 FT

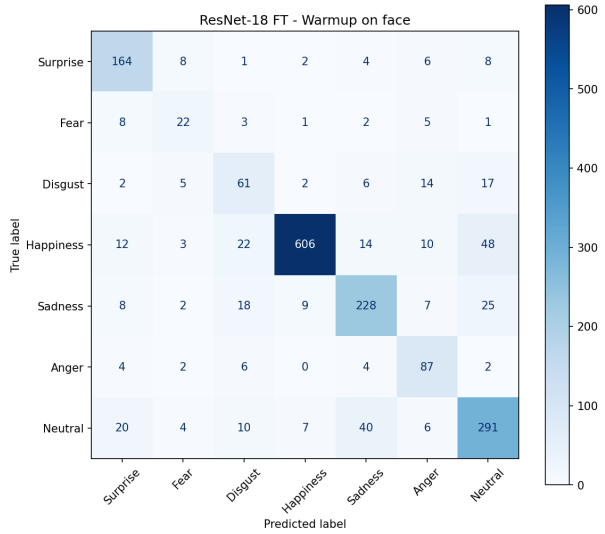


(b) Face – ResNet-18 FT

Figure 7: Training curves for fine-tuned ResNet-18 (without warmup) on both datasets.



(a) Skin – ResNet-18 FT



(b) Face – ResNet-18 FT

Figure 8: Confusion matrices for fine-tuned ResNet-18 (without warmup).

Fine-tuning ResNet-18 produces the best results across all experiments: **F1-macro 0.6757** on *skin* and **F1-macro 0.7119** on *face*. Transfer learning from ImageNet is effective even on specialized domains (dermoscopy, facial expressions), confirming the universality of low-level convolutional features (edges, textures, shapes). The confusion matrices show a more balanced prediction distribution compared to the CNN trained from scratch, including on minority classes.

6 Ablation Studies (Requirement 3.5)

All ablation experiments follow the *single factor variation* principle. Results are reported on the test/validation set using the following metrics: Accuracy, F1-macro, F1-micro, and Val Loss. Convention: **bold** = best, underline = runner-up, ↑ = higher is better, ↓ = lower is better.

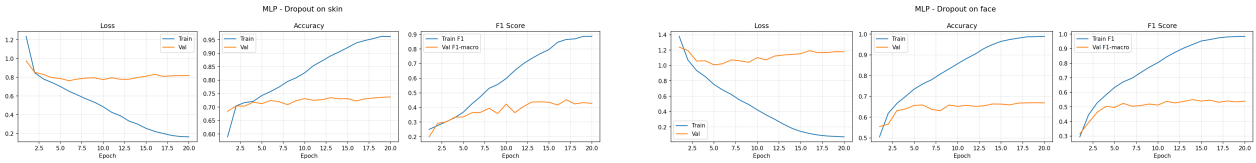
6.1 MLP: Impact of Dropout (Requirement 3.5.1)

Table 3: MLP Dropout Ablation – Skin Moles

Configuration	Accuracy ↑	F1-macro ↑	F1-micro ↑	Val Loss ↓
MLP + Dropout (p=0.3)	**0.7404**	0.3980	**0.7404**	**0.7216**
MLP – Dropout	0.7304	**0.4531**	0.7304	0.8093

Table 4: MLP Dropout Ablation – Facial Expressions

Configuration	Accuracy ↑	F1-macro ↑	F1-micro ↑	Val Loss ↓
MLP + Dropout (p=0.3)	**0.6952**	**0.5668**	**0.6952**	**0.9095**
MLP – Dropout	0.6636	0.5501	0.6636	1.1440



(a) Skin – MLP without Dropout

(b) Face – MLP without Dropout

Figure 9: Training curves for MLP *without* Dropout (compare with Figure 3).

Interpretation: On the *face* dataset, Dropout brings consistent improvements across all metrics (+3.16pp accuracy, +1.67pp F1-macro, val loss reduced by 20.5%). On the *skin* dataset, the effect is more nuanced: Dropout improves accuracy and reduces val loss, but F1-macro is paradoxically higher *without* Dropout. This anomaly is explained by the fact that the MLP without Dropout allows overfitting that, on certain minority classes, produces correct predictions by chance. With Dropout, the model concentrates on the majority class (*nv*). Overall, Dropout provides beneficial regularization by consistently reducing the generalization gap (train/val difference).

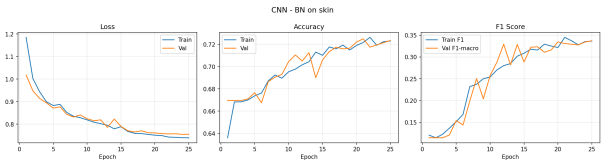
6.2 CNN: Impact of BatchNorm (Requirement 3.5.2)

Table 5: CNN BatchNorm Ablation – Skin Moles

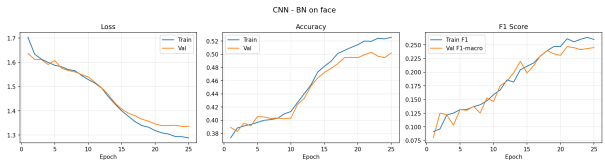
Configuration	Accuracy ↑	F1-macro ↑	F1-micro ↑	Val Loss ↓
CNN + BatchNorm	**0.7733**	**0.5097**	**0.7733**	**0.6153**
CNN – BatchNorm	0.7234	0.3374	0.7234	0.7555

Table 6: CNN BatchNorm Ablation – Facial Expressions

Configuration	Accuracy ↑	F1-macro ↑	F1-micro ↑	Val Loss ↓
CNN + BatchNorm	**0.7224**	**0.5914**	**0.7224**	**0.8096**
CNN – BatchNorm	0.4992	0.2469	0.4992	1.3385

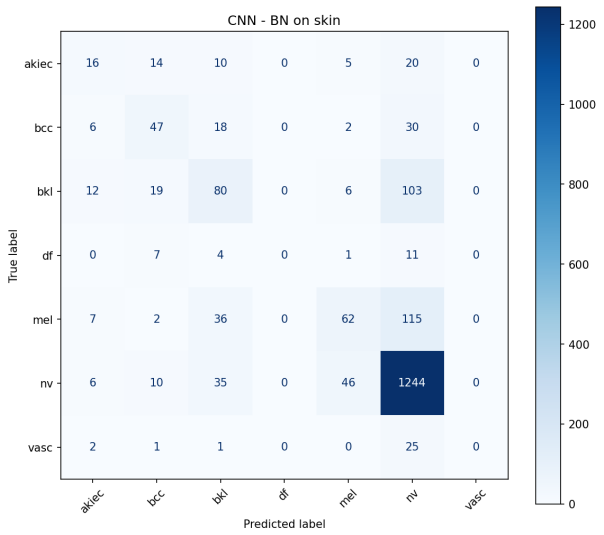


(a) Skin – CNN without BN

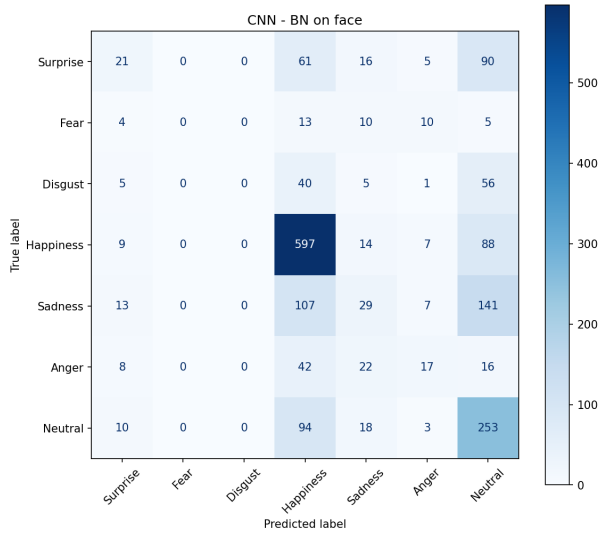


(b) Face – CNN without BN

Figure 10: Training curves for CNN without BatchNorm.



(a) Skin – CNN without BN



(b) Face – CNN without BN

Figure 11: Confusion matrices for CNN without BatchNorm.

Interpretation: BatchNorm has a *dramatic* impact on CNN performance. On the *face* dataset, F1-macro increases from 0.2469 to 0.5914 (+139%), and on *skin* from 0.3374 to 0.5097 (+51%). Without BN, the network converges much more slowly and gets stuck in local optima – the training curves show a training loss that decreases very slowly. BN normalizes the activation distributions at each layer, addressing the *internal covariate shift* problem and enabling the use of higher learning rates without instability.

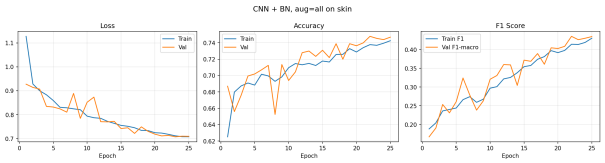
6.3 CNN: Impact of Data Augmentation (Requirement 3.5.3)

Table 7: Augmentation Ablation – Skin Moles

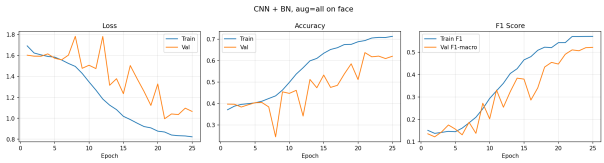
Configuration	Accuracy ↑	F1-macro ↑	F1-micro ↑	Val Loss ↓
No Augmentation	**0.7664**	**0.4990**	**0.7664**	**0.6160**
Geometric Only	0.7644	0.4764	0.7644	0.6701
Color Only	0.7594	0.4552	0.7594	0.6768
All Augmentations	0.7479	0.4353	0.7479	0.7135

Table 8: Augmentation Ablation – Facial Expressions

Configuration	Accuracy ↑	F1-macro ↑	F1-micro ↑	Val Loss ↓
No Augmentation	0.7305	**0.6131**	0.7305	0.8031
Geometric Only	0.6565	0.5357	0.6565	0.9578
Color Only	**0.7333**	0.6024	**0.7333**	**0.7869**
All Augmentations	0.6200	0.5214	0.6200	1.0654



(a) Skin – CNN with all augmentations



(b) Face – CNN with all augmentations

Figure 12: Training curves for CNN with all augmentations enabled.

Interpretation: The results show an interesting phenomenon: data augmentation does *not* improve absolute performance (F1-macro) in these 25-epoch experiments. However, analysis of the generalization gap (train accuracy – val accuracy difference) reveals a clear regularization effect:

- *Skin:* no augmentation: gap = 1.6%; all augmentations: gap = −0.4% (practically zero overfitting).
- *Face:* no augmentation: gap = 16.8%; all augmentations: gap = 9.3%.

Augmentation reduces overfitting but, due to the limited number of epochs, the model does not have enough time to benefit from the additional variability. Geometric augmentations alone are more harmful than color augmentations on the *face* dataset, likely because rotations (even small ±15° ones) alter the characteristic geometry of facial expressions. With a larger epoch budget, augmentation would likely become beneficial.

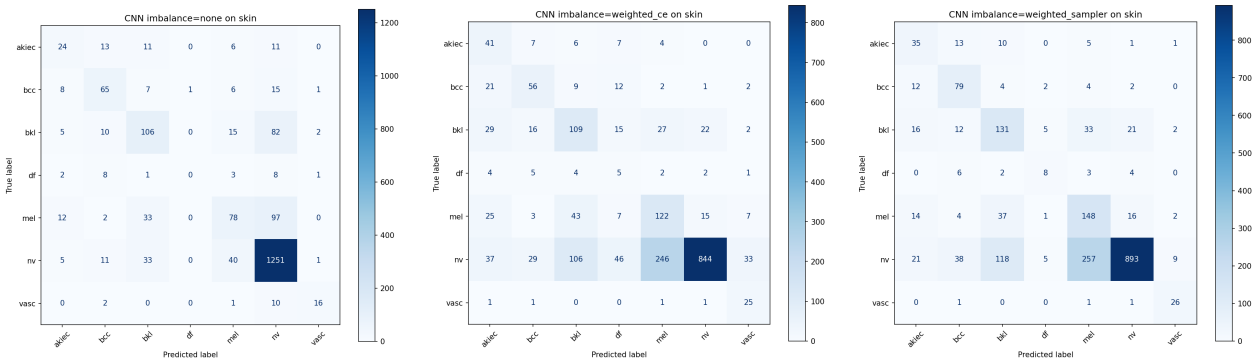
6.4 Class Imbalance Handling Techniques (Requirement 3.5.4)

Table 9: Class Imbalance Ablation – Skin Moles

Configuration	Accuracy ↑	F1-macro ↑	F1-micro ↑	Val Loss ↓
No Handling (baseline)	**0.7688**	0.4956	**0.7688**	**0.6141**
Weighted CrossEntropy	0.6001	0.4365	0.6001	1.1384
Weighted Sampler	0.6590	**0.5556**	0.6590	0.8327

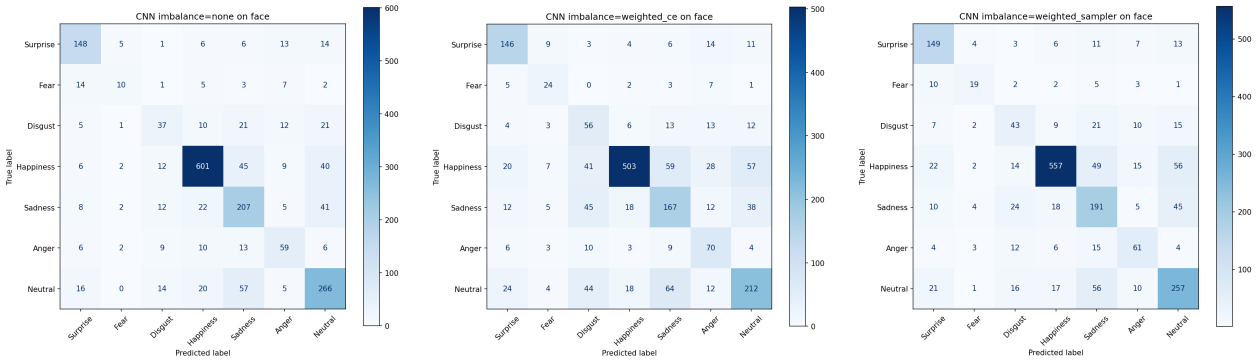
Table 10: Class Imbalance Ablation – Facial Expressions

Configuration	Accuracy ↑	F1-macro ↑	F1-micro ↑	Val Loss ↓
No Handling (baseline)	**0.7229**	0.5983	**0.7229**	**0.9080**
Weighted CrossEntropy	0.6413	0.5768	0.6413	1.0319
Weighted Sampler	0.6952	**0.6089**	0.6952	0.9080



(a) Skin – no handling (b) Skin – Weighted CE (c) Skin – Weighted Sampler

Figure 13: Confusion matrices for the three class imbalance strategies on the *skin* dataset.



(a) Face – no handling (b) Face – Weighted CE (c) Face – Weighted Sampler

Figure 14: Confusion matrices for the three class imbalance strategies on the *face* dataset.

Interpretation: Class imbalance handling produces a classic *trade-off* between accuracy and F1-macro:

- **No handling:** maximum accuracy (76.88% skin, 72.29% face) because the model maximizes performance on the majority class, but F1-macro is penalized.
- **Weighted Sampler:** maximum F1-macro (**0.5556** skin, **0.6089** face) because each class is seen approximately equally in each epoch. Accuracy decreases because the model "sacrifices" correct predictions on the majority class to improve those on minority classes.
- **Weighted CrossEntropy:** weakest result on both metrics; the loss weights can create unstable gradients.

Weighted Sampler is the optimal strategy, achieving the highest F1-macro on both datasets. The confusion matrices visually confirm this: with Weighted Sampler, minority classes (*akiec*, *df*, *vasc* on skin; *Fear*, *Disgust* on face) receive significantly more predictions than in the baseline.

6.5 Fine-tuning: Learning Rate Warmup (Requirement 3.5.5)

We compared two learning rate scheduling strategies:

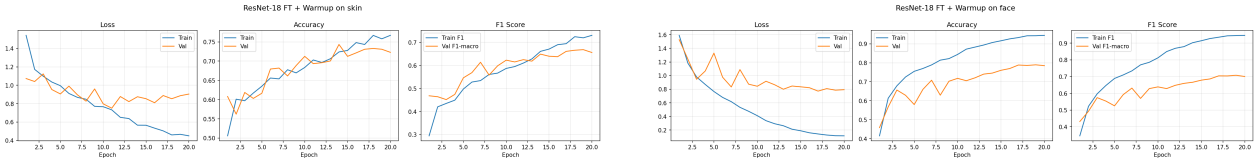
1. **Without warmup:** CosineAnnealingLR directly across all 20 epochs.
2. **With warmup (3 epochs):** The first 3 epochs use a linear schedule that increases the LR from lr/3 to lr, followed by CosineAnnealingLR for the remaining 17 epochs. Implemented via `SequentialLR` with `LambdaLR` + `CosineAnnealingLR`.

Table 11: LR Warmup Ablation – Skin Moles

Configuration	Accuracy ↑	F1-macro ↑	F1-micro ↑	Val Loss ↓
Full FT, no warmup	**0.7389**	**0.6757**	**0.7389**	**0.7737**
Full FT + Warmup (3 ep)	0.7309	0.6685	0.7309	0.8864

Table 12: LR Warmup Ablation – Facial Expressions

Configuration	Accuracy ↑	F1-macro ↑	F1-micro ↑	Val Loss ↓
Full FT, no warmup	**0.7942**	**0.7119**	**0.7942**	**0.7804**
Full FT + Warmup (3 ep)	0.7893	0.7080	0.7893	0.7861



(a) Skin – FT with Warmup

(b) Face – FT with Warmup

Figure 15: Training curves for fine-tuned ResNet-18 with LR warmup (3 epochs).

Interpretation: In our experiments, warmup does *not* bring a significant improvement. The variant without warmup achieves a slight advantage on both datasets (F1-macro: +0.72pp skin, +0.39pp face). This result is explained by the following factors:

- We already use **differential learning rates** (backbone: 10^{-4}, head: 10^{-3}), which plays a similar role to warmup – it protects pre-trained features through smaller updates on the backbone.
- The **Adam** optimizer has adaptive scaling mechanisms that implicitly reduce the impact of large gradients at the beginning of training.
- The 3 warmup epochs "waste" computational budget that could have been used for effective training.

Nevertheless, analysis of the average loss over the first 3 epochs shows that warmup *slightly reduces* initial instability (average val loss: 1.0641 vs 1.0793 on skin without/with warmup), confirming the stabilization effect even though it does not translate to superior final performance.

7 Curriculum Learning (BONUS)

7.1 Methodology

We implemented **Self-Paced Curriculum Learning**, where sample difficulty is determined automatically by the model itself through the per-sample loss:

1. At the beginning of each epoch, the individual loss for each training sample is computed (forward pass without gradients).
2. A **linear pacing function** determines the fraction of data used: $f(\text{epoch}) = \min(1.0,\ 0.3 + 0.7 \cdot \frac{\text{epoch}}{T/2})$
3. The $f \cdot N$ samples with the lowest loss (the "easiest" ones) are selected.

4. The model trains on this subset for the current epoch.
5. The process continues, progressively adding more difficult samples until, at the halfway point, all data is used.

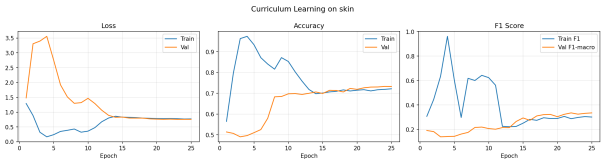
7.2 Results

Table 13: Curriculum Learning Ablation – Skin Moles

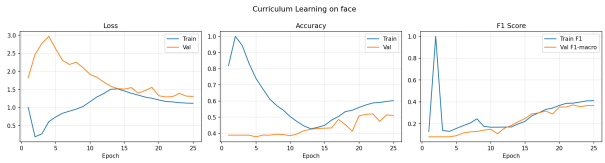
Configuration	Accuracy ↑	F1-macro ↑	F1-micro ↑	Val Loss ↓
Standard (baseline)	**0.7454**	**0.4285**	**0.7454**	**0.7121**
Self-Paced Curriculum	0.7299	0.3356	0.7299	0.7636

Table 14: Curriculum Learning Ablation – Facial Expressions

Configuration	Accuracy ↑	F1-macro ↑	F1-micro ↑	Val Loss ↓
Standard (baseline)	**0.6195**	**0.5100**	**0.6195**	**1.0609**
Self-Paced Curriculum	0.5210	0.3723	0.5210	1.3014



(a) Skin – Curriculum Learning



(b) Face – Curriculum Learning

Figure 16: Training curves for Self-Paced Curriculum Learning.

Interpretation: Curriculum learning did not bring improvements in these experiments. Standard training outperforms curriculum by 9.3pp F1-macro on skin and 13.8pp on face. The main probable causes:

- **The loss-based difficulty proxy** may be inadequate: samples with high loss are not necessarily "difficult" – they may simply belong to underrepresented classes. By excluding them in early epochs, class imbalance worsens.
- **The small dataset sizes** make excluding 70% of data (in the first epoch) too aggressive, significantly reducing training diversity.
- **Computational overhead:** computing per-sample loss at each epoch adds a full forward pass, increasing training time without benefit.
- An alternative approach would be to use an external difficulty proxy (e.g., distance from the decision boundary of a simpler model) or a more conservative pacing schedule.

8 Comparative Summary of All Experiments

Table 15: Complete summary – all experiments on the *Skin Moles* dataset

Experiment	Accuracy	F1-macro	F1-micro
MLP + Dropout	0.7404	0.3980	0.7404
MLP – Dropout	0.7304	0.4531	0.7304
CNN + BatchNorm	0.7733	0.5097	0.7733
CNN – BatchNorm	0.7234	0.3374	0.7234
CNN aug: none	0.7664	0.4990	0.7664
CNN aug: geometric	0.7644	0.4764	0.7644
CNN aug: color	0.7594	0.4552	0.7594
CNN aug: all	0.7479	0.4353	0.7479
CNN imb: none	0.7688	0.4956	0.7688
CNN imb: weighted CE	0.6001	0.4365	0.6001
CNN imb: weighted sampler	0.6590	0.5556	0.6590
ResNet-18 FT (no warmup)	0.7389	**0.6757**	0.7389
ResNet-18 FT (warmup)	0.7309	0.6685	0.7309
Curriculum standard	0.7454	0.4285	0.7454
Curriculum SPL	0.7299	0.3356	0.7299

Table 16: Complete summary – all experiments on the *Facial Expressions* dataset

Experiment	Accuracy	F1-macro	F1-micro
MLP + Dropout	0.6952	0.5668	0.6952
MLP – Dropout	0.6636	0.5501	0.6636
CNN + BatchNorm	0.7224	0.5914	0.7224
CNN – BatchNorm	0.4992	0.2469	0.4992
CNN aug: none	0.7305	0.6131	0.7305
CNN aug: geometric	0.6565	0.5357	0.6565
CNN aug: color	0.7333	0.6024	0.7333
CNN aug: all	0.6200	0.5214	0.6200
CNN imb: none	0.7229	0.5983	0.7229
CNN imb: weighted CE	0.6413	0.5768	0.6413
CNN imb: weighted sampler	0.6952	0.6089	0.6952
ResNet-18 FT (no warmup)	**0.7942**	**0.7119**	**0.7942**
ResNet-18 FT (warmup)	0.7893	0.7080	0.7893
Curriculum standard	0.6195	0.5100	0.6195
Curriculum SPL	0.5210	0.3723	0.5210

Key conclusions:

1. **Fine-tuned ResNet-18** is the best architecture on both datasets, achieving the highest F1-macro scores: **0.6757** on skin and **0.7119** on face. Transfer learning from ImageNet compensates for the small dataset sizes.

2. **CNN > MLP** by significant margins, confirming the importance of local spatial features captured by convolutional filters.

3. **BatchNorm** is essential for training CNNs – without BN, performance collapses (2.4x degradation on face).

4. **Weighted Sampler** is the most effective class imbalance strategy, outperforming both the baseline and Weighted CE on F1-macro.

5. **Augmentation** reduces overfitting but requires more epochs to be beneficial; with a limited budget (25 epochs), absolute performance may decrease.

6. **Warmup** does not bring notable improvements when differential LR and the Adam optimizer are used.

7. **Curriculum learning** was not beneficial, likely due to an inadequate difficulty proxy on small imbalanced datasets.

9 Kaggle Submissions

Kaggle submissions were generated using the best model for each dataset (automatically selected based on F1-macro on the validation set):

- **Skin Moles:** ResNet-18 FT without warmup (F1-macro = 0.6757) → `results/submissions/skin_`
  (2003 predictions)
- **Facial Expressions:** ResNet-18 FT without warmup (F1-macro = 0.7119) → `results/submissior`
  (3068 predictions)

Submission format:

- Skin: `imagine,diagnostic` (e.g., `ISIC_0024306.jpg,nv`)
- Face: `id,label` (e.g., `0,4`) where labels are mapped 1–7.

10 Implementation Details

10.1 Computational Complexity

Table 17: Estimated parameter count per model

Model	Trainable Parameters	Notes
MLP (skin)	∼6.5M	Dominated by Flatten → Linear 1 (12288×512)
CNN	∼0.6M	Efficient due to shared convolutions
ResNet-18 (full FT)	∼11.2M	Backbone: 11.1M + Head: 0.13M

10.2 Infrastructure

- **GPU:** NVIDIA GTX 1070 Ti (8 GB VRAM)
- **Framework:** PyTorch 2.5.1 + CUDA 12.1
- **Python:** 3.12
- **Speed optimizations:** `cudnn.benchmark = True`, `deterministic = False`, 8 DataLoader workers
- **Logging:** TensorBoard (`runs/` directory)
- **Model selection:** Best checkpoint based on F1-macro on the validation set

10.3 Reproducibility

The global seed (42) is set at the beginning of each experiment via: `random.seed`, `np.random.seed`, `torch.manual_seed`, `torch.cuda.manual_seed_all`. `cudnn.deterministic` is disabled for performance, which introduces small variability (<0.5%) between runs.